INTERNAL ENGINEERING DOCUMENTATION

ACU Routing API

Deployment, Operations & Developer Manual

A complete operational reference covering server initialization, Docker architecture, day-to-day deployment of code and frontend changes, database backup & disaster recovery, and incident troubleshooting for the ACU Routing API platform.

> **Confidential — Internal Use Only.** This document contains server access information and operational procedures for production infrastructure. Do not share outside the engineering/operations team. See Section 13 (Security & Credential Hygiene) before distributing or storing this file.

DOCUMENT OWNER	Engineering / DevOps Team
SYSTEM	ACU Routing API (Flask + PostgreSQL + Docker)
ENVIRONMENT	Ubuntu Server (Production)
VERSION	1.0
LAST UPDATED	July 2026

Table of Contents

1. Introduction & System Architecture

1.1 What this system is

The **ACU Routing API** is a Flask-based backend that manages product routing data (inventory items, production activities, production lines), user authentication/RBAC, an audit log, and a revision/archive history for every product change. It is served in production by **Waitress** (a pure-Python WSGI server) and backed by **PostgreSQL**. A static frontend (a separate repository) is served from the same process using **WhiteNoise**. The whole stack is containerized with **Docker Compose**, which also runs a dedicated backup container that performs scheduled database dumps.

1.2 Technology stack

LAYER	TECHNOLOGY	NOTES
Web framework	Flask 3.0	Application factory in `app.py`
Production server	Waitress	Run via `server.py` / `python server.py`
Static frontend hosting	WhiteNoise	Serves files from `frontend/asd`
Database	PostgreSQL 18	Run as the `db` Docker service
ORM / SQL	SQLAlchemy 2.0 (Core + ORM)	Core for most routes, ORM for users
Auth	JWT (PyJWT) + Argon2id	See `routes/utils/auth_utils.py`
Rate limiting	Flask-Limiter	In-memory, configured in `extension.py`
API docs	Flasgger (Swagger UI)	Available at `/docs/`
Containerization	Docker + Docker Compose	`Dockerfile`, `docker-compose.yml`
Backups	Alpine + cron + pg_dump	`backup/` folder

1.3 Repository layout

The API repository (referred to below as `centralized_routing_api`) contains:

```
centralized_routing_api/
├── app.py                  # Flask application factory
├── server.py                # Waitress production entrypoint
├── config.py                 # Reads all settings from environment variables
├── db.py                     # SQLAlchemy Core engine/pool
├── extension.py               # Flask-Limiter instance
├── requirements.txt
├── dockerfile
├── docker-compose.yml
├── init-db/01-init.sql        # Schema + seed data, run on first DB boot
├── backup/                    # Backup container (Dockerfile, crontab, backup.sh)
├── routes/
│   ├── __init__.py            # Blueprint registration
│   ├── auth.py, items.py, update.py, logs.py,
│   │   production_lines.py, archive.py, export.py, health.py
│   ├── models.py              # ORM User model + scoped session
│   └── utils/                 # decorators, auth_utils, log_utils, archive_utilities
└── frontend/                  # NOT included in this repo — see Section 4.3
    └── asd/                   # Pulled separately from its own GitHub repo
```

> **NOTE**
>
> The `frontend/` folder does not exist by default when you clone this repository. It must be created and populated manually — this is covered in Section 4.3 and is one of the most important steps in setting up a new server, because the app will not serve the website (only the raw API) until this folder exists.

1.4 Request flow overview

An external request hits Waitress on the configured port (default `5000`). Waitress wraps the Flask app with WhiteNoise so any request that matches a static file under `frontend/asd` is served directly; everything else is routed to Flask, which resolves it to one of the registered blueprints (`/api/...`). Flask handlers use SQLAlchemy to talk to the `db` Postgres container over the internal Docker network. Nginx/reverse proxy is not part of this repository — Waitress is exposed directly on the mapped port.

2. Prerequisites & Access Requirements

2.1 Accounts & tools you need

- SSH access to the production Ubuntu server (see Section 3).
- Git access to the **ACU Routing API** repository (backend code).
- Git access to the **frontend** repository: `https://github.com/Dev1ze32/asd.git`
- Docker & Docker Compose installed on the server (Section 4.1).
- An SSH client on your own machine (OpenSSH, PuTTY, or a terminal app).
- A copy of the current production `.env` file, or the values needed to recreate it (see Section 6). Never store this file in Git.

2.2 Important note on where code is written

> **YOU DO NOT DEVELOP ON THE PRODUCTION SERVER**
>
> Because this application is deployed on a bare Ubuntu server (no desktop environment, no IDE), **all code changes must be written and tested locally on a developer's machine first**, then pushed to GitHub, and finally pulled onto the server for deployment. The server is a deployment/runtime target only — it is not a coding environment. Never edit files directly on the server with `nano` / `vim` as a substitute for a real change; if you must patch something urgently, still commit that patch to Git afterwards so the repository stays the source of truth.

3. Connecting to the Production Server (SSH)

3.1 Server connection details

Current production server access (rotate immediately if this document is ever shared beyond the team — see Section 13):

```
Host / IP       192.168.11.207
Username        prod_db
Password        Pai@SCM127#
Protocol        SSH (port 22, default)
```

> **DANGER — CONFIDENTIAL CREDENTIAL**
>
> The password above grants shell access to the production server. Treat it exactly like a production database password: do not paste it into chat tools, tickets, screenshots, or personal notes outside of this controlled document. See Section 13 for the rotation procedure — it is strongly recommended to move to SSH key authentication and retire this password entirely.

3.2 Connecting from Windows / macOS / Linux

Windows (PowerShell / Windows Terminal, OpenSSH built-in)

```
ssh prod_db@192.168.11.207
```

You will be prompted for the password above. If `ssh` is not recognized, enable the "OpenSSH Client" optional Windows feature, or use PuTTY: Host Name = `192.168.11.207`, Port = `22`, Connection type = SSH.

macOS / Linux (Terminal)

```
ssh prod_db@192.168.11.207
# type the password when prompted (it will not be visible as you type)
```

First-time connection warning

On the very first connection you will see a message similar to:

```
The authenticity of host '192.168.11.207' can't be established.
ED25519 key fingerprint is SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type `yes` and press Enter. This is expected the first time you connect to a new server and simply adds the server's key to your local `known_hosts` file.

3.3 Hardening recommendations

RECOMMENDED (DO THIS ONCE, THEN USE GOING FORWARD)

1. Generate an SSH key pair on your machine: `ssh-keygen -t ed25519 -C "yourname@company"`
2. Copy your public key to the server: `ssh-copy-id prod_db@192.168.11.207`
3. Confirm key-based login works: `ssh prod_db@192.168.11.207` (should no longer ask for a password)
4. Once every team member who needs access has a key installed, disable password authentication in `/etc/ssh/sshd_config` (`PasswordAuthentication no`) and restart `sshd` .
5. Consider creating individual named accounts per engineer (with sudo/docker-group access) instead of everyone sharing the `prod_db` login, for auditability.

4. First-Time Server Initialization

Follow this section only when setting up a **brand-new server** (or rebuilding one from scratch). If the server is already running, skip to Section 7 (Deploying Updates).

4.1 Install Docker & Docker Compose

```
# Update package index
sudo apt update && sudo apt upgrade -y

# Install Docker Engine (official convenience script)
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Allow your user to run docker without sudo (log out/in after this)
sudo usermod -aG docker $USER

# Verify installation
docker --version
docker compose version
```

4.2 Clone the API repository

```
cd /opt        # or wherever you keep deployed applications
git clone <YOUR_API_REPO_URL> centralized_routing_api
cd centralized_routing_api
```

4.3 Pull in the frontend (required, separate repo)

DO NOT SKIP THIS STEP

The Python server (`server.py`) is hard-coded to serve the website's static files from `frontend/asd` , relative to the repository root:

```
root=os.path.join(os.path.dirname(__file__), "frontend", "asd")
```

This folder is **not included** in the API repository by default. On a brand new checkout, the folder does not exist, and the site will not load (only the raw `/api/...` endpoints will respond) until you pull it in manually.

From the root of the API repo (`centralized_routing_api/`):

```
# 1. Create the frontend directory
mkdir -p frontend

# 2. Clone the frontend repository INTO the frontend folder
cd frontend
git clone https://github.com/Dev1ze32/asd.git

# 3. Go back to the repo root
cd ..

# 4. Confirm the resulting path is correct
ls frontend/asd
```

Because the repository is named `asd`, cloning it while inside the `frontend/` folder automatically creates `frontend/asd/`, which is exactly the path `server.py` expects. Do not rename this folder.

> **REMINDER**
>
> This is one repo pull, not a build step by itself — if the frontend has its own build process (e.g. a bundler that outputs static HTML/JS/CSS), run that build so the compiled output actually lands in `frontend/asd` before continuing to Section 4.5. Check that repository's own README for its exact build command if one exists.

4.4 Create the environment file (.env)

The application never hard-codes secrets — everything is read from environment variables via `config.py` (loaded from a `.env` file at the repo root using `python-dotenv`). Create it now:

```
cd centralized_routing_api
nano .env
```

Paste in and fill out the following (see Section 6 for a full explanation of every variable):

```
# ── PostgreSQL ──
DB_HOST=db
DB_PORT=5432
DB_NAME=routing_db
DB_USER=postgres
DB_PASSWORD=<choose a strong password>

# ── JWT ──
JWT_SECRET_KEY=<generate with: openssl rand -hex 32>
JWT_ACCESS_TOKEN_EXPIRES_HOURS=24

# ── Initial admin account seed (first boot only) ──
INITIAL_ADMIN_USERNAME=admin
INITIAL_ADMIN_PASSWORD=<choose a strong password>

# ── Server ──
WAITRESS_PORT=5000
WAITRESS_THREADS=8
```

> **NEVER COMMIT .ENV**
>
> `.env` is already listed in `.gitignore` and `.dockerignore`. Keep it that way. Store a securely-shared backup copy of production values in your team's password manager, not in Git, chat, or plain-text files on shared drives.

4.5 Build and start the stack

```
docker compose build
docker compose up -d
```

This starts three containers in order (Compose waits for the database health check before starting the API — see Section 5.1):

- `acu-routing-db` — PostgreSQL, running the schema in `init-db/01-init.sql` on its very first boot only.
- `acu-routing-api` — the Flask/Waitress application, serving both the API and the frontend.
- `acu-routing-backup` — the scheduled backup container (see Section 8).

4.6 Verify the deployment

```
# Check all three containers are "Up" / healthy
docker compose ps

# Tail the API logs to confirm a clean startup
docker compose logs -f api

# Hit the health endpoint from the server itself
curl http://localhost:5000/api/health
# Expected: {"status": "ok"}
```

Then, from your own browser, visit `http://192.168.11.207:5000/` — you should see the frontend website, and `http://192.168.11.207:5000/docs/` should show the interactive Swagger API documentation.

5. Docker Architecture Deep Dive

5.1 docker-compose.yml services

The stack is defined in `docker-compose.yml` at the repository root and declares three services on one internal Docker network (`internal`):

SERVICE	CONTAINER NAME	IMAGE / BUILD	PURPOSE
`db`	acu-routing-db	`postgres:18` (official image)	Database engine; runs SQL in `init-db/` on first boot only.
`api`	acu-routing-api	Built from repo root `Dockerfile`	Flask/Waitress app; serves API + frontend static files.
`backup`	acu-routing-backup	Built from `backup/Dockerfile`	Cron-driven `pg_dump` backups (weekly + monthly).

The `api` service depends on `db` being **healthy** (`pg_isready` check) before it starts, so a fresh `docker compose up -d` never races the database. The API port is published to the host via:

```
ports:
  - "0.0.0.0:${WAITRESS_PORT:-5000}:${WAITRESS_PORT:-5000}"
```

Only this port is exposed to the outside world — the database is only reachable from inside the `internal` Docker network (it is declared with `expose` , not `ports`), so it cannot be reached directly from the internet.

5.2 The api image (`Dockerfile`)

The image is built in two stages to keep the final runtime image small:

- **Stage 1 (builder):** a full `python:3.12-slim` image with `gcc` and `libpq-dev` , used only to compile `psycopg[binary]` / `argon2-cffi` and install everything from `requirements.txt` into an isolated prefix (`/install`).
- **Stage 2 (runtime):** a fresh `python:3.12-slim` image with only the lightweight runtime library `libpq5` , the installed packages copied over from the builder, and the application source (`COPY --chown=appuser:appuser . .`). It runs as a non-root user (`appuser`), never as root.

The container's entrypoint is `python server.py` , and Docker's own `HEALTHCHECK` hits `/api/health` every 30 seconds — three consecutive failures marks the container "unhealthy", which Compose will restart automatically because of `restart: unless-stopped` .

5.3 The db service & the `postgres_data` volume

PostgreSQL's own data directory inside the container (`/var/lib/postgresql`) is mounted to a **named Docker volume** called `postgres_data`, declared at the bottom of `docker-compose.yml`:

```
volumes:
  postgres_data:/var/lib/postgresql   # inside the db service
…
volumes:
  postgres_data:                      # top-level named volume declaration
```

> **THIS IS THE SINGLE MOST IMPORTANT THING TO UNDERSTAND ABOUT THIS STACK**
>
> `postgres_data` is a **Docker-managed named volume**, not a plain folder inside your project directory. Docker stores its actual contents under its own storage path (typically `/var/lib/docker/volumes/<project>_postgres_data/_data` on the host) — it is **not** part of the Git repository and is **not** something `git clone` / `git pull` ever touches. This is what protects the live database from ordinary code deployments. However, it also means the volume is easy to accidentally destroy with the wrong Docker command — see Section 9 for the specific dangers and how to avoid them.

5.4 The backup service

`backup/Dockerfile` builds a minimal Alpine image containing the PostgreSQL client tools, `gzip`, and `cron`. It copies in `backup.sh` and a `crontab` file, and its container command starts `crond` in the background, then tails the log file forever so `docker compose logs` shows backup activity. This is the service — and only this service — that reads database credentials and dumps the database on a schedule (Section 8).

5.5 Networking

All three containers join a single Compose-managed bridge network named `internal`. Containers reach each other by service name (e.g. the API's `DB_HOST` is literally the string `db`, which Docker's embedded DNS resolves to the database container's internal IP). Nothing besides the API's published port is reachable from outside the Docker host.

6. Environment Variables Reference

All configuration is centralized in `config.py` and loaded from `.env` (or the container's environment, in the case of the `api` service, whose `env_file: .env` plus `DB_HOST: db` override is set directly in `docker-compose.yml`).

VARIABLE	DEFAULT	DESCRIPTION
`DB_HOST`	`localhost`	Set to `db` in Docker (overridden automatically by Compose).
`DB_PORT`	`5432`	PostgreSQL port.
`DB_NAME`	`routing_db`	Database name.
`DB_USER`	`postgres`	Database user.
`DB_PASSWORD`	`postgres`	**Must** be overridden in production. Required by Compose (build fails otherwise).
`DB_POOL_SIZE`	20	Persistent DB connections kept open (≈ threads).
`DB_MAX_OVERFLOW`	10	Extra burst connections allowed.
`DB_POOL_TIMEOUT`	10	Seconds to wait for a free connection before a 503.
`DB_POOL_RECYCLE`	1800	Recycle connections older than this (seconds).
`DB_CONNECT_TIMEOUT`	5	TCP handshake timeout for new connections.
`DB_STATEMENT_TIMEOUT_MS`	30000	Postgres cancels queries running longer than this.
`JWT_SECRET_KEY`	*dev placeholder*	**Must** be set in production. If missing, a random key is generated at every process start, which silently logs everyone out on every restart/deploy.
`JWT_ACCESS_TOKEN_EXPIRES_HOURS`	24	Login session length.
`RATE_LIMIT_LOGIN`	10/minute	Per-IP login attempt limit.
`RATE_LIMIT_REGISTER`	5/minute	Per-IP account-creation limit.
`RATE_LIMIT_DEFAULT`	2000/minute	Global default limit for all other endpoints.
`WAITRESS_THREADS`	8	Worker thread pool size for Waitress.
`WAITRESS_PORT`	5000	Port the app listens on (also used in the Docker port mapping).
`INITIAL_ADMIN_USERNAME` / `INITIAL_ADMIN_PASSWORD`	—	Only used on a completely empty `users` table, to seed the first admin account.

TIP

Generate a strong JWT secret with: `openssl rand -hex 32` . Once set, avoid changing it casually in production — changing it invalidates every currently-logged-in user's session immediately.

7. Deploying Code Changes & New Features

7.1 Standard backend update workflow

This is the normal path for shipping a bug fix or a small change to the API:

1. **Locally:** make your change, test it, commit, and push to the repository's main/deploy branch on GitHub.
2. **SSH into the server** (Section 3) and move into the repo:

```
ssh prod_db@192.168.11.207
cd /opt/centralized_routing_api
```

3. **Pull the latest code:**

```
git pull origin main
```

4. **Rebuild and restart only the API container** (the database keeps running and its data is untouched, since it lives in the separate `postgres_data` volume, not the image):

```
docker compose build api
docker compose up -d api
```

5. **Watch the logs** to confirm a clean startup:

```
docker compose logs -f api
```

6. **Smoke-test** the health endpoint and a couple of key pages/routes:

```
curl http://localhost:5000/api/health
```

> **ZERO-DOWNTIME NOTE**
>
> This project does not currently run multiple API replicas behind a load balancer, so `docker compose up -d api` will cause a brief (typically a few seconds) interruption while the old container stops and the new one starts and passes its health check. Schedule routine deploys for low-traffic periods when possible.

7.2 Deploying frontend changes

The frontend lives in its own repository (`https://github.com/Dev1ze32/asd.git`) inside `frontend/asd/`. To update it:

```
cd /opt/centralized_routing_api/frontend/asd
git pull origin main   # or the frontend repo's actual default branch
# if the frontend has a build step, run it here before continuing
cd /opt/centralized_routing_api

# Rebuild the api image so the new static files get copied into it, then restart
docker compose build api
docker compose up -d api
```

> **FRONTEND FILES ARE BAKED INTO THE IMAGE AT BUILD TIME**
>
> Because the Dockerfile does `COPY --chown=appuser:appuser . .`, the contents of `frontend/asd` at the moment you run `docker compose build` are what gets shipped — the container does not read live from disk afterwards (WhiteNoise's `autorefresh=True` setting only re-reads files *inside the running container*, not from the host). You must rebuild the image any time the frontend changes, exactly the same as for a backend change.

7.3 Adding a new backend feature (blueprint pattern)

This codebase uses one Flask Blueprint per resource. To add a new feature/endpoint group:

1. Create a new file under `routes/`, e.g. `routes/my_feature.py`, following the pattern used in `routes/items.py` — import `managed_connection` from `db.py` (or `managed_db_session` from `routes/models.py` for ORM/User work), wrap your route bodies in `with managed_connection() as conn:`, and protect write endpoints with `@require_auth`, `@require_role(...)`, `@require_superuser_or_admin`, or `@require_admin` as appropriate (see `routes/utils/decorators.py`).
2. Call `log_action(...)` (from `routes/utils/log_utils.py`) after any meaningful write, so it shows up in the audit trail (Section on Logs in the API).
3. Register your new blueprint in `routes/__init__.py`:

   ```
   from routes.my_feature import my_feature_bp
   …
   app.register_blueprint(my_feature_bp)
   ```

4. Add Flasgger-style docstrings to your route functions (see any existing route for the exact YAML format) so the new endpoints automatically appear in `/docs/`.
5. Test locally against a local/dev Postgres instance *before* pushing — never write a new endpoint straight against production data.
6. Push, then follow the standard deploy workflow in Section 7.1.

7.4 Database schema changes

This project does not use a migration framework (e.g. Alembic). The ORM's `Base.metadata.create_all(engine, checkfirst=True)` in `app.py` will create any *new* table that

doesn't yet exist on startup, but it will **not** alter existing tables (add/drop/rename columns) or change constraints.

> **MANUAL SCHEMA CHANGES REQUIRED**
>
> For any change to an existing table (new column, changed constraint, new index on a Core-managed table like `products` / `activities` / `activity_logs` / `product_revisions` / `production_lines` / `line_activities`, which are all defined only in `init-db/01-init.sql`, not in the ORM), you must:
>
> 1. Write a plain SQL migration statement (e.g. `ALTER TABLE products ADD COLUMN ...`).
> 2. Take a fresh backup first (Section 8.3).
> 3. Apply it directly against the running database:
>
>    ```
>    docker compose exec db psql -U postgres -d routing_db -c "ALTER TABLE products ADD
>    ```
>
> 4. Update `init-db/01-init.sql` in the repo to match, so a brand-new server initializes with the same schema going forward (that file only runs automatically on a completely empty database volume).

7.5 Rollback procedure

If a deploy introduces a regression:

```
# 1. Find the last known-good commit
git log --oneline -n 10

# 2. Check out that commit (or revert the bad commit)
git checkout <good-commit-hash>
# or: git revert <bad-commit-hash> && git push

# 3. Rebuild and restart exactly as in a normal deploy
docker compose build api
docker compose up -d api
```

Application code rollbacks are safe and fast because they never touch the database. If the regression also changed the database schema, you may additionally need to restore from backup (Section 10) — schema rollbacks are riskier and should be planned carefully.

8. Database Backups

8.1 How automatic backups work

The `backup` container runs `cron` against the schedule defined in `backup/crontab`:

SCHEDULE	RUNS	COMMAND
Every Sunday at 1:00 AM	Weekly backup	`/app/backup.sh weekly`
1st of every month at 2:00 AM	Monthly backup	`/app/backup.sh monthly`

`backup.sh` runs `pg_dump` against the `db` service over the internal Docker network, pipes the output through `gzip`, and writes a timestamped file. It logs everything to `/var/log/backup.log` inside the container, which is also what `docker compose logs backup` shows.

8.2 Where backups are stored

DOCKER CREATES THIS FOLDER FOR YOU AUTOMATICALLY

`docker-compose.yml` mounts a host-side *bind mount* for the backup service:

```
volumes:
  - ./backups:/backups
```

Unlike `postgres_data`, this **is** a plain folder inside the project directory (`centralized_routing_api/backups/`). You do not need to create it manually — Docker creates `./backups` (and its `weekly/` and `monthly/` subfolders) on the host automatically the first time the backup container writes to it. Because it lives on the host filesystem, it survives `docker compose down` and even a full container/image rebuild.

```
centralized_routing_api/
└── backups/
    ├── weekly/
    │   └── routing_db_20260628_010000.sql.gz
    └── monthly/
        └── routing_db_20260601_020000.sql.gz
```

8.3 Taking a manual backup

Run this any time before a risky operation (schema change, major deploy, restore test):

```
docker compose exec backup /app/backup.sh weekly
# or: docker compose exec backup /app/backup.sh monthly

# Confirm the file exists
ls -la backups/weekly/
```

8.4 Retention policy

TYPE	KEPT FOR	CLEANUP MECHANISM
Weekly backups	90 days	`find ... -mtime +90 -exec rm -f {} \;` in `backup.sh`
Monthly backups	360 days	`find ... -mtime +360 -exec rm -f {} \;` in `backup.sh`

Old files are pruned automatically by `backup.sh` itself, right after each successful run of the same type. Application-level audit logs are pruned separately and manually via the `DELETE /api/logs/cleanup` endpoint (admin-only, default 90 days — see `routes/logs.py`); this is unrelated to database file backups.

> **OFF-SERVER COPIES**
>
> The `backups/` folder lives on the same physical/virtual server as the database it's backing up. For real disaster recovery (server loss, disk failure), you should periodically copy the contents of `backups/` to a separate location (another server, cloud storage, an external drive) — this repository does not do that for you automatically.

9. Critical: Protecting the Production Database

9.1 Never copy a local `postgres_data` folder to/from the server

Some engineers, while developing locally, temporarily switch `docker-compose.yml` to use a **bind mount** instead of a named volume (e.g. `./postgres_data:/var/lib/postgresql`) so they can easily inspect or reset their local database files. If you ever do this locally, you will end up with a literal `postgres_data/` folder sitting inside your local copy of the project.

> **THE DANGER**
>
> If that local `postgres_data/` folder is ever included when you copy, zip, `rsync`, or otherwise transfer your project directory onto the production server (for example, "just uploading the whole folder" instead of using Git), and the server's compose configuration is later pointed at that same path, you can silently **overwrite or corrupt the live production database** with your local test data — with no warning and no easy way to tell it happened until users start reporting missing or wrong data.

9.2 Safe ways to transfer files

> **RULES TO FOLLOW**
>
> - **Only use `git pull`** to update code on the server. Never `scp` / `rsync` /upload your whole local project folder to the server as a shortcut.
> - Add `postgres_data/` to your local `.gitignore` if you ever create it locally, so it can never accidentally be committed or bundled.
> - If you must transfer a specific non-data file (e.g. a one-off config), name it explicitly in the copy command — never copy an entire directory tree that could contain a `postgres_data` or stray `.env` file.
> - The production `docker-compose.yml` should always use the named volume (`postgres_data:` with no leading `./`), never a host bind mount, for the database. Do not "fix a local dev annoyance" by changing this file and pushing that change — keep any local bind-mount override in an uncommitted `docker-compose.override.yml` instead.
> - Before running any command on the server that you are not 100% sure about, run a manual backup first (Section 8.3).

9.3 Commands that are dangerous on production

COMMAND	WHY IT IS DANGEROUS
`docker compose down -v`	**Deletes the named volumes**, including `postgres_data` — this destroys the entire production database. Only ever use plain `docker compose down` (no `-v`) on production.
`docker volume rm centralized_routing_api_postgres_data`	Directly deletes the database volume. Never run manually unless you have just confirmed, from a fresh backup, that this is intentional.
`docker system prune -a --volumes`	Wipes unused images *and volumes* across the whole Docker host — extremely destructive; never run this on a production host without first confirming exactly what it would remove (`docker system prune -a --volumes --dry-run` is not supported by all Docker versions, so inspect `docker volume ls` manually first).
Copying a full project directory over the server instead of `git pull`	Can overwrite `.env`, drag in a local `postgres_data` bind mount, or reset file permissions unexpectedly (see 9.1).

10. Restoring From a Backup

10.1 Before you restore

> **RESTORING OVERWRITES CURRENT DATA**
>
> A restore replaces the live database contents with the contents of the backup file. Any data written after the backup was taken will be lost unless you take one more "just-in-case" backup of the current (possibly broken) state before proceeding.

1. Confirm you have identified the correct file in `backups/weekly/` or `backups/monthly/` (check the timestamp in the filename).
2. Take a safety backup of the current state, even if it's the thing you're trying to fix: `docker compose exec backup /app/backup.sh weekly`
3. Notify the team / put up a maintenance notice if this is a live production restore — the API will be briefly unavailable.

10.2 Restore procedure

```
# 1. Stop the API so nothing writes to the database mid-restore
docker compose stop api

# 2. Pick your backup file
BACKUP_FILE=backups/weekly/routing_db_20260628_010000.sql.gz

# 3. Drop and recreate the database (inside the db container) so the
#    restore starts from a clean slate
docker compose exec db psql -U postgres -c "DROP DATABASE IF EXISTS routing_db;"
docker compose exec db psql -U postgres -c "CREATE DATABASE routing_db;"

# 4. Stream the compressed backup back in
gunzip -c $BACKUP_FILE | docker compose exec -T db psql -U postgres -d routing_db

# 5. Start the API again
docker compose start api
```

10.3 Post-restore verification

```
# Health check
curl http://localhost:5000/api/health

# Confirm row counts look reasonable
docker compose exec db psql -U postgres -d routing_db -c "SELECT COUNT(*) FROM products;"
docker compose exec db psql -U postgres -d routing_db -c "SELECT COUNT(*) FROM users;"

# Log in through the frontend / Swagger UI to confirm auth still works
```

> **NOTE ON JWT SESSIONS AFTER A RESTORE**
>
> Restoring the database does not by itself invalidate JWT tokens (the secret key is separate, in `.env`), so users who were logged in before the restore may still have valid tokens referencing user IDs that behave normally as long as those user rows still exist in the restored data.

11. Troubleshooting & Incident Runbook

11.1 First response checklist

- [ ] Is the server itself reachable? `ping 192.168.11.207` / try SSH.
- [ ] Are all containers running? `docker compose ps`
- [ ] Does the health endpoint respond? `curl http://localhost:5000/api/health`
- [ ] Check the most recent logs for a stack trace: `docker compose logs --tail=200 api`
- [ ] Check disk space (a full disk breaks Postgres and Docker builds): `df -h`
- [ ] Check whether a deploy happened recently (`git log -1`) that might correlate.

11.2 Common issues & fixes

SYMPTOM	LIKELY CAUSE	FIX
Site returns raw JSON errors instead of the website / frontend 404s	`frontend/asd` is missing or empty in the built image	Confirm `frontend/asd` exists and is populated (Section 4.3), then `docker compose build api && docker compose up -d api`.
`503` with `"code": "db_pool_exhausted"`	Too many concurrent DB connections; pool timeout hit	Check for a runaway/long query: `docker compose exec db psql -U postgres -d routing_db -c "SELECT pid, now()-query_start AS age, state, query FROM pg_stat_activity ORDER BY age DESC LIMIT 10;"`. Consider raising `DB_POOL_SIZE` / `DB_MAX_OVERFLOW` in `.env` if load has genuinely grown.
`503` with `"code": "db_unavailable"`	Postgres container is down, restarting, or unreachable	`docker compose ps` and `docker compose logs db`. Restart with `docker compose restart db` if it crashed; investigate disk space first.
`429 Too Many Requests`	Flask-Limiter rate limit hit (login: 10/min, register: 5/min, default: 2000/min)	Expected behavior under abuse/brute-force; if a legitimate office IP is hitting default limits, raise `RATE_LIMIT_DEFAULT` in `.env` and restart the API.
All users get logged out after every deploy/restart	`JWT_SECRET_KEY` is not set (or still the dev placeholder) in `.env`, so a new random secret is generated on every process start (a loud warning is logged about this — search logs for `JWT_SECRET_KEY`)	Set a permanent `JWT_SECRET_KEY` in `.env` (Section 6) and restart the API once — after that it will be stable across restarts.
API container keeps restarting / crash-looping	Startup exception (bad `.env` value, DB unreachable, syntax error in a recently deployed file)	`docker compose logs api` and read the traceback near the bottom. If it's a bad deploy, roll back (Section 7.5).
`docker compose build` fails compiling `psycopg` / `argon2-cffi`	Corrupted build cache, or a transient network issue pulling packages	`docker compose build --no-cache api`
Backup container shows no recent backups	`cron` not running, or `backup.sh` failing silently	`docker compose logs backup` to see cron/pg_dump output; test manually with `docker compose exec backup /app/backup.sh weekly`.

New database column/table added in code doesn't appear	Only ORM-managed tables auto-create; Core-managed tables need manual SQL (Section 7.4)	Apply the SQL manually via `docker compose exec db psql ...` and update `init-db/01-init.sql`.

11.3 Reading logs

```
# Follow live logs for one service
docker compose logs -f api
docker compose logs -f db
docker compose logs -f backup

# Last N lines only, no follow
docker compose logs --tail=100 api

# All services at once
docker compose logs -f
```

Application-level activity (who did what) is also queryable directly through the API for admins: `GET /api/logs` (paginated, filterable by user/action/date — see `routes/logs.py`), backed by the `activity_logs` table.

11.4 Full application restart

```
# Restart just the API (fast, database untouched)
docker compose restart api

# Restart everything (rare — only if the DB itself is misbehaving)
docker compose down
docker compose up -d
## NEVER add -v to the "down" command above — see Section 9.3
```

12. Routine Maintenance Checklist

Daily

- ☐ Glance at `docker compose ps` — all three containers should read "Up".
- ☐ Spot-check `curl http://localhost:5000/api/health`.

Weekly

- ☐ Confirm the Sunday 1 AM backup landed in `backups/weekly/`.
- ☐ Review `docker compose logs --tail=300 api` for recurring warnings/errors.
- ☐ Check disk usage: `df -h` and `docker system df`.

Monthly

- ☐ Confirm the 1st-of-month backup landed in `backups/monthly/`.
- ☐ Copy the latest backup off-server to a secondary location (Section 8.4).
- ☐ **Do a test restore** into a throwaway local/staging environment to verify backups are actually restorable, not just present.
- ☐ Review the audit log (`GET /api/logs`) for unexpected admin actions.
- ☐ `docker image prune` to clean up old, unused build layers (safe — does not touch named volumes).

Quarterly

- ☐ Rotate the `prod_db` SSH password (or confirm key-only auth is enforced).
- ☐ Review who has server/Git access and remove anyone who no longer needs it.
- ☐ Run `sudo apt update && sudo apt upgrade` on the Ubuntu host for OS security patches (schedule a maintenance window).

13. Security & Credential Hygiene

> **ACTION ITEM FROM THIS DOCUMENT**
>
> The SSH password documented in Section 3.1 has now been written down in this manual and should be treated as no more secret than any other internal document. If this file is ever emailed, uploaded to a shared drive, or handed to anyone outside the immediate ops/engineering team, **rotate the `prod_db` password immediately** (`passwd prod_db` on the server) and move to SSH key authentication (Section 3.3) as soon as possible so passwords are no longer the access mechanism at all.

General principles

- **Never commit secrets to Git.** `.env`, database dumps, and any file containing passwords/keys must stay out of version control (already covered by `.gitignore` / `.dockerignore` — do not remove those entries).
- **Principle of least privilege.** Only admins should hold `admin`-role application accounts; day-to-day users should be `user` or `superuser` as appropriate (see the RBAC roles in `routes/models.py`).
- **Rotate `JWT_SECRET_KEY` only when necessary** (e.g. suspected compromise) — rotating it force-logs-out every user.
- **Keep the database unreachable from the internet.** It is already only exposed on the internal Docker network (Section 5.1) — do not add a `ports:` mapping for the `db` service.
- **Use a firewall on the Ubuntu host** (e.g. `ufw`) to only allow SSH (22) and the application port (5000, or 80/443 if fronted by a reverse proxy in the future) from where they're actually needed.
- **Keep dependencies current.** Periodically review `requirements.txt` for outdated/CVE-affected packages.

14. Appendix A — Command Cheat Sheet

Everyday

```
docker compose ps                       # status of all containers
docker compose logs -f api              # follow API logs
docker compose restart api              # restart API only (DB untouched)
curl http://localhost:5000/api/health   # health check
```

Deploying

```
git pull origin main
docker compose build api
docker compose up -d api
```

Backups

```
docker compose exec backup /app/backup.sh weekly
docker compose exec backup /app/backup.sh monthly
ls -la backups/weekly/ backups/monthly/
```

Database shell / manual SQL

```
docker compose exec db psql -U postgres -d routing_db
```

Full (safe) restart of the whole stack

```
docker compose down        # NEVER add -v on production
docker compose up -d
```

Cleaning up disk space (safe)

```
docker image prune                # removes dangling/unused images only
docker system df                  # see what's using space
```

Things to NEVER run on production

```
docker compose down -v                              # deletes the database volume
docker volume rm centralized_routing_api_postgres_data
docker system prune -a --volumes
```

15. Appendix B — Directory & Port Reference

Key paths (on the server, inside `centralized_routing_api/`)

PATH	WHAT IT IS
`.env`	Production secrets/config — never in Git.
`frontend/asd/`	Pulled-in frontend repo — required for the site to render (Section 4.3).
`backups/weekly/`, `backups/monthly/`	Auto-created by Docker; database dump files (Section 8).
`init-db/01-init.sql`	Runs only on a brand-new, empty database volume.
`routes/`	All Flask blueprints (API endpoints).
`backup/`	Backup container source (Dockerfile, crontab, backup.sh).

Ports

PORT	SERVICE	EXPOSED TO
22	SSH	Host network (restrict via firewall/allowlist).
5000 (configurable via `WAITRESS_PORT`)	API + frontend (Waitress)	Published to the host / internet.
5432	PostgreSQL	Internal Docker network only — not published.

Key URLs once running

URL	PURPOSE
`http://<server-ip>:5000/`	Frontend website
`http://<server-ip>:5000/api/health`	Health check (used by Docker's own HEALTHCHECK too)
`http://<server-ip>:5000/docs/`	Swagger / interactive API documentation
`http://<server-ip>:5000/api/auth/login`	Login endpoint

End of document — ACU Routing API Deployment, Operations & Developer Manual, v1.0, July 2026.
Keep this document updated as the system evolves; treat it as living documentation, not a one-time write-up.